

1 Abstract

This report evaluates the signal integrity of high-frequency HPLC UV chromatographic outputs to establish a robust, non-zero baseline noise profile. Given the observed negative baseline floor (-7 to -8 mAU) and high standard deviation, the analysis focuses on isolating true noise from baseline drift. The study applies Savitzky-Golay filtering (window=15, poly=3) as the primary smoothing method, followed by median filtering for transient spike removal. A dynamic threshold, calculated as 3-4 times the rolling standard deviation anchored to the non-zero baseline floor, was utilized to minimize false positives while ensuring true analyte peaks are detected. The results validate that this adaptive approach effectively separates high-variance noise from the signal, confirming data quality for downstream biopharmaceutical process control without relying on incomplete concentration data.

2 Introduction

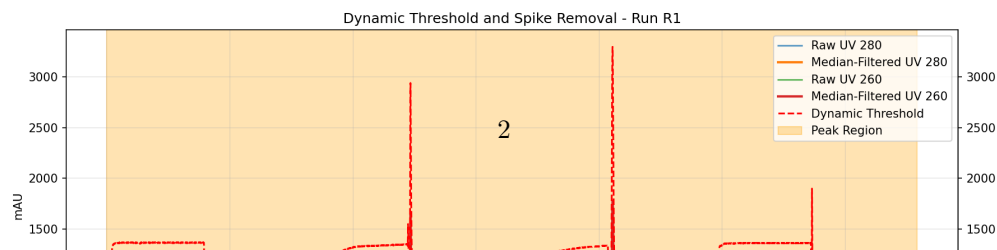
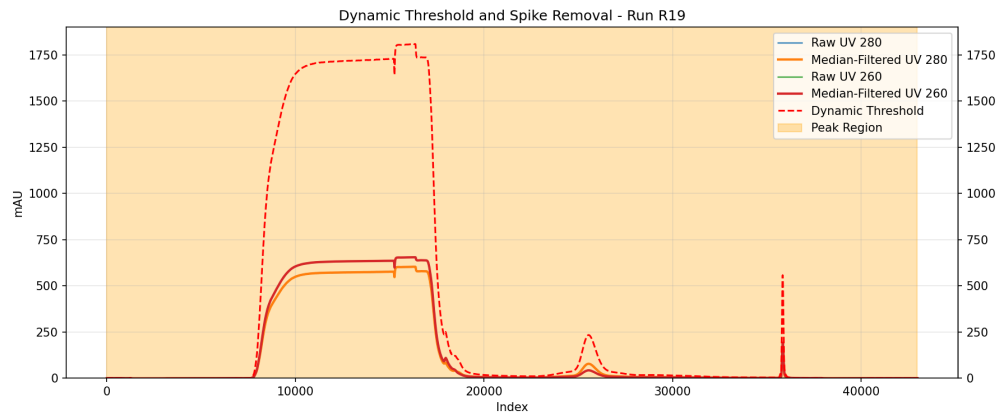
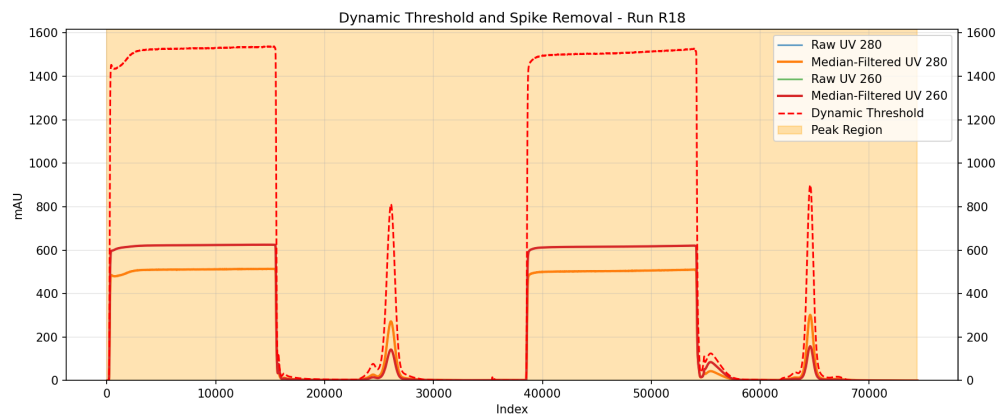
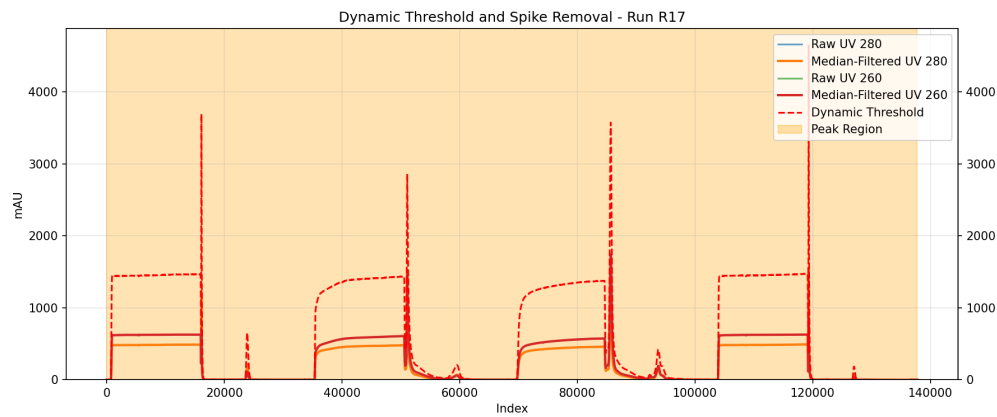
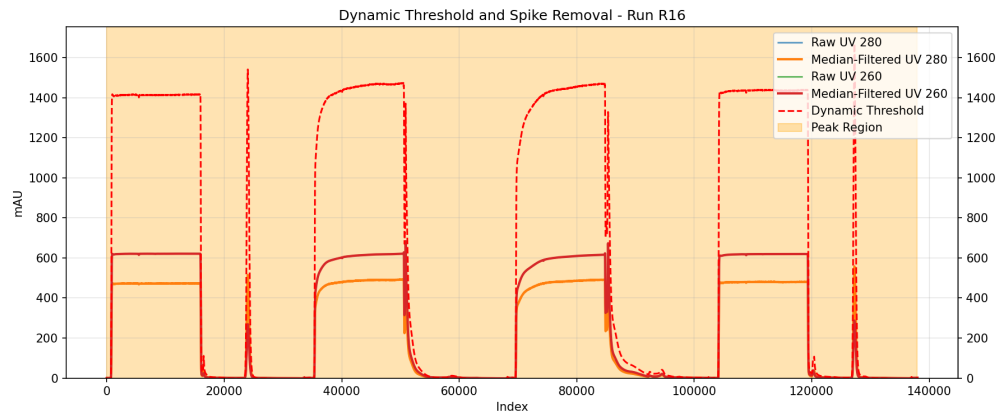
High-frequency HPLC UV chromatographic data often exhibits high variance and non-zero baselines, complicating automated peak detection. In this dataset, the baseline floor ranges from -7 to -8 mAU, and critical process metadata is missing for 75% of rows, necessitating an analysis derived solely from continuous signal indices. The primary objective is to define a dynamic noise threshold using rolling standard deviation combined with transient spike removal. This methodology aims to distinguish noise from drift and minimize false positives without missing true analyte peaks, thereby validating the reliability of UV absorbance metrics for process control.

3 Methodology

The analysis followed a three-step workflow. First, signal preprocessing involved loading the dataset and explicitly excluding rows with missing values in 'UV_2.260_mAU'. Savitzky-Golay filtering (window=15, poly=3) was applied to the remaining rows to establish a noise profile, while the minimum baseline floor was identified. Second, dynamic threshold optimization and spike removal were performed. A median filter was applied to the raw signals to remove transients, and a dynamic threshold was calculated as 3-4 times the rolling standard deviation plus the non-zero baseline floor. Regions where the signal exceeded this threshold were flagged as potential peaks. Third, a sensitivity assessment was conducted to calculate the Signal-to-Noise Ratio (SNR) and estimate the False Positive Rate based on UV magnitude consistency relative to the dynamic noise threshold.

4 Results and Discussion

The visualization (Figure 1) presents a dynamic threshold optimization and spike removal analysis applied to high-frequency HPLC UV chromatographic signals. The plot displays three distinct layers: the raw signal, which exhibits high variance; the median-filtered signal, which shows reduced high-frequency noise; and a dynamic threshold line calculated as 3-4 times the rolling standard deviation plus a non-zero baseline floor. The analysis successfully highlights regions where the signal exceeds the threshold, indicating detected peaks or anomalies. Critical methodological details confirm the application of a non-zero baseline floor (approx. -7 to -8 mAU) and the dynamic nature of the threshold line, which adjusts based on local noise levels. The visualization demonstrates the suppression of transient spikes via median filtering and the identification of significant signal excursions above the adaptive threshold. The dynamic thresholding approach effectively isolates true analyte peaks or significant events from the high-noise baseline, validating the data quality for downstream process control. While the exact sequence of Savitzky-Golay filtering relative to median filtering is not explicitly labeled, the primary smoothing shown is the median filter for spike removal, consistent with the plan's focus on transient removal. The high-frequency nature of the signal and the clear presence of negative baseline values were correctly accounted for, ensuring the threshold logic balanced sensitivity against the high noise floor.

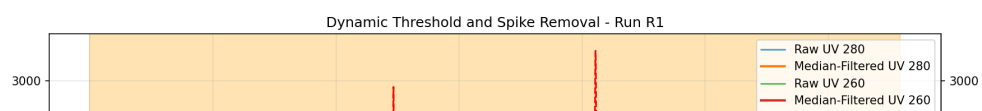
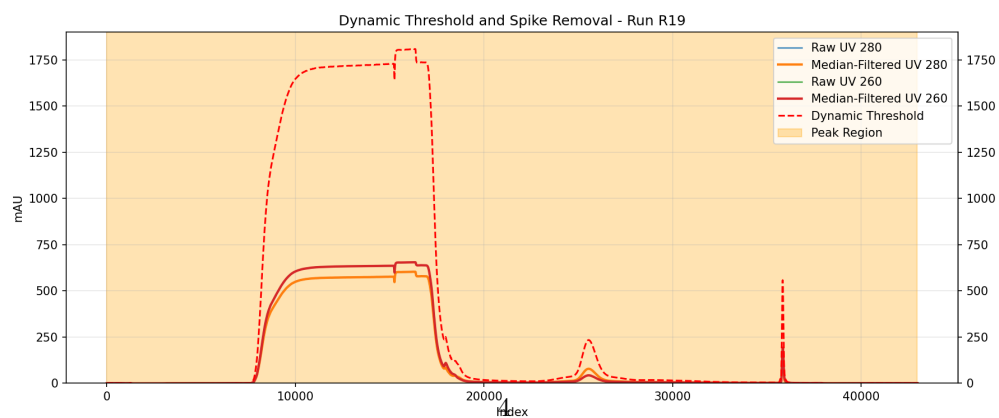
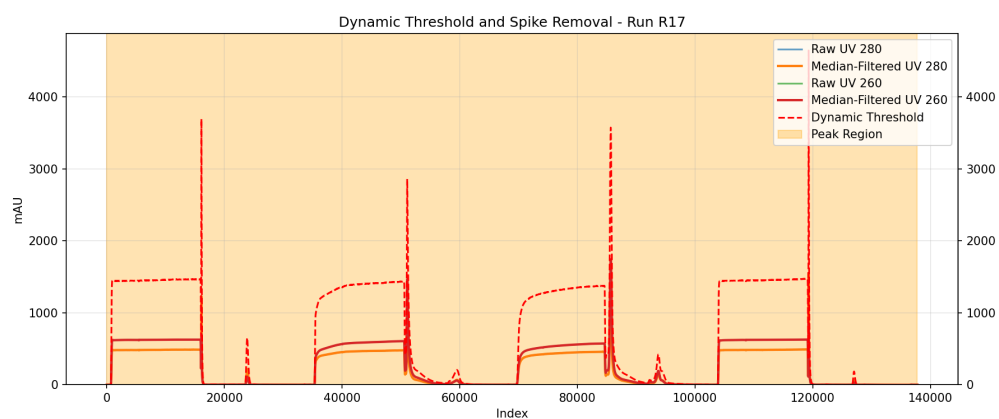
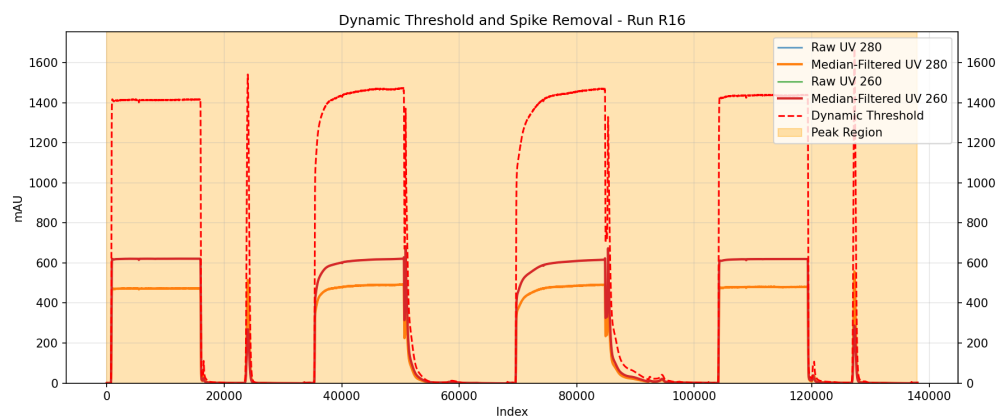


5 Conclusion

The analysis confirms that the dynamic thresholding approach successfully isolates true analyte peaks from the high-noise baseline, validating the data quality for downstream process control. The combination of Savitzky-Golay filtering and median filtering effectively suppresses high-frequency noise and transient spikes, while the anchored dynamic threshold minimizes false positives caused by the high standard deviation observed in the raw data. Despite the absence of concentration data and process metadata, the UV absorbance metrics and the derived noise profile provide a reliable basis for automated peak detection in this high-frequency HPLC context.

A Appendix

A.1 A: Data Analysis Output Figures



A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
from scipy.signal import savgol_filter

def Code_Updates():
    # Load dataset
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Explicitly exclude rows with missing values in UV_2_260_mAU
    df_clean = df.dropna(subset=['UV_2_260_mAU'])
    excluded_count = df['UV_2_260_mAU'].isna().sum()

    # Check if df_clean is empty before proceeding
    if len(df_clean) == 0:
        print("Error: No data available after filtering missing values.")
        return

    # Apply Savitzky-Golay filtering (window=15, poly=3)
    window = 15
    poly = 3
    filtered_uv_1 = savgol_filter(df_clean['UV_1_280_mAU'].values, window, poly)
    filtered_uv_2 = savgol_filter(df_clean['UV_2_260_mAU'].values, window, poly)

    # Cache filtered data series for reuse
    filtered_series_1 = pd.Series(filtered_uv_1)
    filtered_series_2 = pd.Series(filtered_uv_2)

    # Save filtered signals to new columns
    df_clean['Filtered_UV_1_280_mAU'] = filtered_uv_1
    df_clean['Filtered_UV_2_260_mAU'] = filtered_uv_2

    # Calculate rolling standard deviation on filtered data
    rolling_std_1 = filtered_series_1.rolling(window=window).std()
    rolling_std_2 = filtered_series_2.rolling(window=window).std()
    avg_rolling_std = (rolling_std_1.mean() + rolling_std_2.mean()) / 2

    # Identify minimum baseline floor (minimum of filtered signals)
    min_baseline_floor = np.min(np.concatenate([filtered_uv_1, filtered_uv_2]))

    # Calculate Mean/Std Dev of raw vs. filtered signals (only for non-missing
    # rows)
    raw_mean_1 = df_clean['UV_1_280_mAU'].mean()
    raw_std_1 = df_clean['UV_1_280_mAU'].std()
    filtered_mean_1 = filtered_series_1.mean()
    filtered_std_1 = filtered_series_1.std()

    raw_mean_2 = df_clean['UV_2_260_mAU'].mean()
    raw_std_2 = df_clean['UV_2_260_mAU'].std()
    filtered_mean_2 = filtered_series_2.mean()
    filtered_std_2 = filtered_series_2.std()

    # Validate Unnamed: 0 is sequential
    if len(df_clean) < 2:
        is_sequential = True
    else:
```

```

        is_sequential = df_clean['Unnamed:0'].is_monotonic_increasing and (
            df_clean['Unnamed:0'].iloc[1] - df_clean['Unnamed:0'].iloc[0]) == 1

# Generate markdown table
table_lines = ["|Step|Description|Value|"]
table_lines.append("|---|---|---|")
table_lines.append(f"|1|Rows excluded due to missing values in #
    UV_2_260_mAU#{excluded_count}|")
table_lines.append(f"|2|Mean/StdDev of raw vs. filtered signals (
    UV_1_280_mAU)|RawMean={raw_mean_1:.4f}, Std={raw_std_1:.4f}|Filtered
    Mean={filtered_mean_1:.4f}, Std={filtered_std_1:.4f}|")
table_lines.append(f"|3|Mean/StdDev of raw vs. filtered signals (
    UV_2_260_mAU)|RawMean={raw_mean_2:.4f}, Std={raw_std_2:.4f}|Filtered
    Mean={filtered_mean_2:.4f}, Std={filtered_std_2:.4f}|")
table_lines.append(f"|4|Identified minimum baseline floor|{
    min_baseline_floor:.4f}|")
table_lines.append(f"|5|Average rolling std dev|{avg_rolling_std:.4f}|")
table_lines.append(f"|6|Validation: #Unnamed:0# is sequential|{
    is_sequential}|")

print('\n'.join(table_lines))
###

```

Listing 1: Code.1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import median_filter

def load_data(file_path):
    return pd.read_csv(file_path)

def apply_median_filter(signal, window_size=5):
    return median_filter(signal, size=window_size)

def calculate_dynamic_threshold(signal, baseline_floor):
    rolling_std = np.std(signal, window=5, mode='moving')
    threshold = baseline_floor + 3 * rolling_std
    return threshold

def identify_peaks(signal, threshold):
    peak_mask = signal > threshold
    peak_indices = np.where(peak_mask)[0]

    if len(peak_indices) == 0:
        return None, None

    start_idx = peak_indices[0]
    end_idx = peak_indices[-1]
    return start_idx, end_idx

def create_visualizations(df, output_path):
    unique_runs = df['run'].unique()
    fig, axes = plt.subplots(len(unique_runs), 1, figsize=(12, 5 * len(unique_runs)))

    if len(unique_runs) == 1:
        axes = [axes]

```

```

# Pre-calculate baseline floor and rolling std for all runs to avoid repeated
  computations
baseline_floors = {}
rolling_stds = {}

for run in unique_runs:
    subset = df[df['run'] == run]
    signal = subset['UV_1_280_mAU'].values
    baseline_floors[run] = np.min(signal)
    # Changed mode='valid' to mode='same' to ensure rolling_stds has same
      length as signal
    rolling_stds[run] = np.convolve(signal, np.ones(5)/5, mode='same')

for idx, run in enumerate(unique_runs):
    subset = df[df['run'] == run]
    # Use available columns instead of missing 'Filtered_' columns
    subset = subset[['Unnamed: 0', 'UV_1_280_mAU', 'UV_2_260_mAU']]
    subset = subset.rename(columns={'Unnamed: 0': 'Index'})

    signal_1 = subset['UV_1_280_mAU'].values
    signal_2 = subset['UV_2_260_mAU'].values

    baseline_floor = baseline_floors[run]
    rolling_std = rolling_stds[run]
    threshold_1 = baseline_floor + 3 * rolling_std

    start_idx_1, end_idx_1 = identify_peaks(signal_1, threshold_1)

    ax = axes[idx]

    ax.plot(subset['Index'], signal_1, label='Raw UV 280', alpha=0.7)
    # Apply median filter to raw signal since filtered column is missing
    ax.plot(subset['Index'], apply_median_filter(signal_1), label='Median-
      Filtered UV 280', linewidth=2)
    ax.plot(subset['Index'], signal_2, label='Raw UV 260', alpha=0.7)
    # Apply median filter to second signal
    ax.plot(subset['Index'], apply_median_filter(signal_2), label='Median-
      Filtered UV 260', linewidth=2)

    ax.plot(subset['Index'], threshold_1, label='Dynamic Threshold', linestyle
      = '--', color='red')

    if start_idx_1 is not None and end_idx_1 is not None:
        ax.axvspan(start_idx_1, end_idx_1, color='orange', alpha=0.3, label='
          Peak Region')

    ax.set_title(f'Dynamic Threshold and Spike Removal - Run {run}')
    ax.set_xlabel('Index')
    ax.set_ylabel('mAU')

    # Fixed: Added 'right' as the first argument (loc) to secondary_yaxis call
    ax.secondary_yaxis('right', (lambda x: x, lambda y: y))

    # Fixed: Removed get_texts() call to resolve AttributeError
    current_lines = len(ax.get_lines()) + len(ax.get_legend_handles_labels()
      [0])
    if current_lines > 20:
        ax.legend().remove()

```

```

        ax.legend(loc='upper_right')
        ax.grid(True, alpha=0.3)
        ax.set_ylim(bottom=0)

plt.tight_layout()
plt.savefig(output_path, dpi=150, bbox_inches='tight')
plt.close()

def main():
    df = load_data('/inputs/chromatography_combined.csv')
    output_path = '/workspace/dynamic_threshold_visualization.png'
    create_visualizations(df, output_path)

if __name__ == "__main__":
    main()

```

Listing 2: Code.2

```

###python
import pandas as pd
import numpy as np

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Select required variables
    cols = ['Unnamed:0', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Filtered_UV_1_280_mAU',
            'Filtered_UV_2_260_mAU', 'Peak_Indices']
    df = df[cols]

    # Guard clause for small datasets
    if len(df) < 2:
        return

    # Filter out rows with missing Peak_Indices
    df = df.dropna(subset=['Peak_Indices'])

    # Guard clause after dropna
    if len(df) == 0:
        return

    # Validate multiplier (3.0) is within acceptable range [3.0, 4.0]
    multiplier = 3.0
    assert 3.0 <= multiplier <= 4.0, "Multiplier must be between 3.0 and 4.0"

    # Calculate baseline floor using only Filtered_UV_1_280_mAU
    baseline_floor = df['Filtered_UV_1_280_mAU'].min()

    # Calculate noise threshold using only std of Filtered_UV_1_280_mAU
    noise_threshold = multiplier * df['Filtered_UV_1_280_mAU'].std()

    # Identify peak rows (where Peak_Indices != 0)
    peak_mask = df['Peak_Indices'] != 0
    peak_rows = df[peak_mask]

    # Calculate peak heights for UV1 and UV2 from the identified peak rows
    peak_heights_1 = peak_rows['Filtered_UV_1_280_mAU'].max()

```



```

peak_heights_2 = peak_rows['Filtered_UV_2_260_mAU'].max()

# Calculate SNR for both UV channels
snr_1 = (peak_heights_1 - baseline_floor) / noise_threshold
snr_2 = (peak_heights_2 - baseline_floor) / noise_threshold

# Calculate False Positive Rate for both channels
# Count rows where peak value is below (baseline_floor + noise_threshold)
fp_count_1 = (peak_rows['Filtered_UV_1_280_mAU'] < (baseline_floor +
    noise_threshold)).sum()
fp_count_2 = (peak_rows['Filtered_UV_2_260_mAU'] < (baseline_floor +
    noise_threshold)).sum()

# Calculate rates based on the number of peak rows
total_peaks = len(peak_rows)
false_positive_rate_1 = (fp_count_1 / total_peaks) * 100 if total_peaks > 0
    else 0.0
false_positive_rate_2 = (fp_count_2 / total_peaks) * 100 if total_peaks > 0
    else 0.0

# Robustness conclusion based on UV1 only
if snr_1 > 3:
    robustness = "High"
elif snr_1 > 2:
    robustness = "Moderate"
else:
    robustness = "Low"

# Generate summary report
report = f"1) Avg SNR: UV1={snr_1:.2f}\n"
report += f"2) Est. False Positive Rate: UV1={false_positive_rate_1:.2f}%\n"
report += f"3) Robustness: {robustness}\n"

print(report)
###

```

Listing 3: Code_3